Two passes over the project as it stands at v3.6.0, written to be useful rather than flattering. The first pass is statistical: is the experimental design sound and do the numbers support the claims. The second is engineering: would this survive contact with people, time, and scale.

Everything below is grounded in a measurement taken tonight or a line of code read tonight. Where a finding was fixed in the same pass, it says so and gives the commit's effect. Where it was not, it says that too.

Headline: the project's top backlog item turned out not to exist, and every defect found here was in the measurement apparatus rather than in the model. The worst of them — read the correction immediately below first — is that the "real games" this project compares itself against were 87% bots, forfeits and stubs.

Every "real" figure in the first draft of this review was drawn from `data/games.ladder.jsonl` read line by line. **That store is 13% usable.** Of 14,453 records: 9,103 bot games, 4,593 partial brings, 3,430 forfeits, 1,950 behavioural bots, 1,177 too short. Only **1,838** are clean.

`engine/quality.js` has always decided this correctly and nearly every other consumer calls it. `realism_report.js` never did, and neither did the mega measurement added the same night. So the comparison was our bots against **other people's bots**, described as realism. That is circular, and it is the single worst error in this document's original version.

Corrected, with the same filter applied to **both** corpora:

metric	first draft "real"	actual clean real	effect on the finding
games containing a mega	93.3%	**98.5%**	our gap is **17 points, worse** than reported
moves super effective	23.3%	**21.4%**	gap 10.2 — §1.4 stands
moves that failed	2.7%	**2.5%**	unchanged, §1.4 stands
turns per game	6.30	**8.35**	gap shrank — bot games are short
switches per game	8.84	**10.67**	gap **grew**; we switch too little after all
distinct sets per species	11.3	**12.2** (ours 12.75)	**§1.3 HOLDS** — still no diversity gap

The mega priors were regenerated from clean games and both moved materially: `p_side_megas` 0.7747 → **0.8801**, `p_mega_is_lead` 0.5770 → **0.4951**. The preview policy tuned in §2.2's sibling commit had therefore been aimed at the wrong targets.

And it is systemic, not one file. A new assertion in `selftest.js` finds **sixteen further engine tools** that name the ladder store and never mention `quality.js`: `backtest_winrate`, `build_mega_dex`, `build_species_abilities`, `chomp-predict`, `coach`, `cores`, `ditto`, `dynamics`, `game-spec`, `illusion`, `ingest_ots`, `kadabra`, `mew_farm`, `playstyle`, `policy`, `validate_selfplay`. Some may legitimately want raw access — none of them says so. **That test is left failing on purpose**, as a tracker rather than an allowlist.

What survives untouched: everything derived from Smogon (§1.8, the 400-sum identity, the 50% rule, the 17% blind spot) and everything that is a property of our own code (§1.1, §1.2, §1.5, §2.1, §2.2). Those never touched the ladder store. The Smogon half of this project rests on 1.16M server-side battles and is by far its most solid foundation.

`build_lab` compared each build to a field mean that **included that build**. The consequence is exact, not approximate: it shrinks every reported difference by `(m-1)/m` and makes the `m` tests more correlated than the Benjamini–Hochberg step assumes.

At `m = 6` that understates every effect by **17%**; at `m = 84`, by 1.2%. So it was invisible in the big runs and material in exactly the small exploratory runs that get iterated on. Leave-one-out costs nothing — `(m·mean - self)/(m-1)` — and is now in place.

The triple loop over moves × items × spreads with a `break` at `--builds N` does not truncate a factorial. It takes a **nested prefix** of one. With 7 move-combos × 4 items × 3 spreads and `--builds 6`, all six arms came back with *identical moves*, and what looked like a factorial was an item/spread study with the move axis frozen.

Factors became confounded with position in the loop, which is the single thing a factorial design exists to prevent. Now every cell is enumerated and walked with a stride coprime to the total near the golden ratio, so a partial run spreads across all three axes — and it prints that it is subsampling. Verified: an 8-arm run now varies all three factors.

`realism_report` capped the generated corpus with `--limit` and read the real corpus **in full**. Distinct-set counts grow with sample size mechanically, so on 292 games against 13,249 it printed 5.7 against 52.0 and flagged itself `<-- large` on every run.

Compared properly — shared species only, first `min(n)` of each — the result **reverses**:

	generated	**real**
distinct sets per species (matched n, 76 spp)	**13.2**	**11.3**

We are slightly *more* varied than the ladder. **There was never a set-diversity gap.** It had been reported three times (23-against-51, then 9.8-against-13.0, then as a thin-candidate-pool problem) and every version was an artifact. The pool explanation was separately disproven: correlation between candidate-pool size and the gap is **0.04** across 28 species.

This is the same class of error that previously invented a switching defect — 23.3 against 46.7 *per 100 moves*, which per game is 4.3 against 5.7. **Before believing any generated-vs-real gap, check that both sides had the same opportunity to produce it.** That rule would have caught all four.

The policy samples moves by usage frequency and does not read the board. Measured against real games:

	generated	**real**
moves that were super effective	10.8%	23.3%
moves that outright failed	8.6%	2.7%
moves that hit an immune target	4.5%	2.2%
turns per game	10.1	6.3

A build that scores well in `build_lab` today may simply be one that suits a bad player. The caveat is printed under every result table, which is honest, but **honesty is not a substitute for validity**. Until the scoring bot exists, `build_lab` answers "which Garchomp suits a pilot that clicks moves by popularity", and nobody wants that answer.

The turns-per-game figure is the quiet one: at 10.1 against 6.3, generated games are structurally longer, so the *positions* being sampled are not drawn from the real distribution either. That matters for anything

trained on them.

`build_lab --species Garchomp` tests Garchomp on **one** team, against **40** opponents. Nothing in the output signals that the conclusion is conditional on both. A build that is better on that team against that field is not a build that is better.

This is the same failure VGC-Bench hit from the other direction — strong on a single fixed team, degrading as team variety rose. The design needs the host team treated as a factor, or at minimum a loud statement of the scope of the claim.

The arms share a gauntlet and (until 1.1) a field mean, so the tests are positively correlated. BH remains valid under positive regression dependence (Benjamini & Yekutieli 2001), so this is not a defect — but it is an assumption being relied on silently and should be stated where the correction is applied.

The table shows a Wilson interval on the raw win rate next to a p-value from the *paired* test. They routinely disagree in appearance: two arms at an identical 72.5% get p = 0.031 and p = 0.087, because the pairing sees different discordant opponents. That is correct behaviour and it looks like a bug. It needs a one-line explanation in the output, or readers will trust the wrong column.

`1 - (1 - other/400)^4 ≈ 17%` of real sets contain a move Smogon does not list individually (median 17%, worst 19%, Kingambit 3%). This is a floor on set realism, it is printed, and it correctly stops anyone chasing the last few points of fidelity. Spreads are worse — Garchomp's spread "Other" is 38.4% against 19.8% for moves — and that is *not* yet surfaced anywhere the user will see it. Minor gap.

Measured, 200 games each:

```
conc=1    14.9 games/sec
conc=4    14.4 games/sec
conc=12   14.3 games/sec
```

The simulator is CPU-bound and single-threaded, so async concurrency *inside one process* buys nothing. All real throughput comes from running multiple **processes**. The 46 games/sec figure this project quotes is a 12-process number.

Two consequences, both live:

- `--conc` looks like a performance control and is not one. It previously caused a misdiagnosis ("scaling collapses past 4 processes") that was entirely an artifact of every measurement using it.
- `set_space.js` **prints "~12.7 hours at 46 games/sec" for the top-14 factorial. That rate requires a 12-process fan-out that does not exist in the tool.** Run as shipped, single process, it is closer to 39 hours. The estimate is not wrong so much as unbuildable from what is there. Either build the fan-out or quote the single-process number.

`engine/selftest.js` **now exists: 17 assertions, no Showdown checkout needed, runs in about a second.** It found a real bug on its first run — `Tatsugiri-Droopy` resolved to nothing and produced an **empty moveset**, the exact silent failure the forme audit was supposed to have closed. (That particular species turned out not to be legal in Champions, so the *test* was wrong; but the fix it prompted is general and the check is now derived rather than hand-listed — see below.)

Two things came out of it:

- `forSpecies` and `resolveSpecies` now strip trailing hyphenated qualifiers progressively, so an unlisted forme falls back to its base species. That is a **rule covering formes nobody has thought of yet**, replacing a hand-kept suffix list — which was itself an S13 violation, and had already failed three times (Vivillon patterns, Floette-Eternal, and this).
- The forme check enumerates **every hyphenated species Smogon reports for this format** instead of five names I picked. Strictly stronger, and it cannot go stale.

The original finding is preserved below because the reasoning still applies to everything not yet covered.

Every fix tonight — the spread forwarding, the leave-one-out, the balanced subsample, the mega preview — was verified by an ad-hoc script written for the occasion and then thrown away. Three of those bugs were *silent*: they produced plausible output while being wrong. Specifically:

- `fillSet` ignored a supplied spread and re-sampled. Detectable only because three "different" arms returned win rates identical to the decimal.
- `build_lab` dropped the spread on the way into `packTeam`. Same tell.
- The results table never printed the spread, which is *why* the tell was needed.

A handful of assertions would have caught all three in a second: *a set built with a forced spread has that spread; two builds differing in a factor produce different packed teams; every varying factor appears in the output*. This is the highest-value engineering work outstanding, and it is small.

`Version 3.5.0` appeared in five separate documents and was bumped tonight with `sed`. The project's own S13 says nothing derivable should be typed. The version should come from one source and be templated in, or the docs will drift from CHANGELOG the first time someone forgets one.

The "item id ends in `ite`" rule now exists in `prior_player.js` and was reimplemented in three throwaway analysis scripts tonight, once *incorrectly* (an anchoring bug that reported zero stones on every team and would have been believed if the number had been merely low instead of absurd). It belongs in one shared module.

`build/publish.sh` did exactly what it claims: size guard, push verification, Pages-deploy verification. It correctly reported "nothing to publish" when the tree was clean and confirmed the deploy URL when it was not. Given that this repo previously went 90 minutes with a broken site while every push reported success, this is the strongest piece of infrastructure in the project.

Spot-checked the Smogon path end to end. `latestMonth()` scans, cutoffs are parameters, and tonight's `set_space.js` replaced the last hand-typed threshold (`LOCK_AT = 85`) with an identity derived from the fact that percentages sum to 400. S12/S13 hold on that path.

---

1. **The scoring bot.** Everything in Pass 1 that is not fixed traces back to it. §1.4 makes every current build-lab number provisional, and §1.5 cannot be answered by a pilot that does not play.
2. **Twenty assertions on the engine** (§2.2). Half a day, and it retires an entire class of silent defect that has now bitten four times.
3. **Either build the multi-process fan-out or stop quoting 46 games/sec** (§2.1). The published cost estimates are currently unachievable with the shipped tools.
4. **Treat the host team as a factor in** `build_lab` (§1.5), or print the scope of the claim.
5. **Version from one source** (§2.3).

- **Do not chase set diversity.** It is closed (§1.3), and we are already more varied than the ladder.

- **Do not tune the mega form-change probability.** Measured: 0.85 → 1.0 moves the rate 0.7 points. The residual 80%-against-93% is a lead/switching issue, downstream of the scoring bot.
- **Do not start the multi-million-game run.** Two of tonight's three statistical defects were in code that produces the analysis, not the data. Burning 12 hours of generation before the analysis layer has assertions is how you get a large corpus you cannot trust.